

PRACTICAL: 05

Aim: Implementation of knapsack algorithm using dynamic programming.

Algorithm:

** Knapsack Algorithm **

Step 1: Start

Step 2: Read n and m

Step 3: Read weight $w[i]$ and value $v[i]$ for each item.

Step 4: Initialize $dp[i][0] = 0$ and $dp[0][j] = 0$

Step 5: for $i = 1$ to n :

for $j = 1$ to w :

if $w[i] \leq j$:

$dp[i][j] = \max(v[i] + dp[i-1][j-w[i]], dp[i-1][j])$

else:

$dp[i][j] = dp[i-1][j]$

Step 6: Max^m value = $dp[n][w]$

Step 7: Set $j = w$

Step 8: for $i = n$ down to 1:

if $dp[i][j] \neq dp[i-1][j]$:

Item i is taken

$j = j - w[i]$

else

Item i is not taken

Step 9: Display max^m value and selected items

Step 10: Stop.

Program / Code:

```
#include <iostream>
```

```
using namespace std;
```

```
class Knapsack {
```

```
private:
```



```
int w[10];      // weights
int v[10];      // values
int n;          // number of items
int W;          // capacity
int dp[10][100]; // DP table
```

public:

// — Get Input —

```
void getInput() {
    cout << "Enter number of items : ";
    cin >> n;

    cout << "Enter knapsack capacity : ";
    cin >> W;

    for (int i = 1; i <= n; i++) {
        cout << "Item " << i << " weight and value : ";
        cin >> w[i] >> v[i];
    }
}
```

// — Build DP Table —

```
void solve() {

    // Base case — row 0 and column 0
    for (int i = 0; i <= n; i++)
```



```
dp[i][0] = 0;

for (int j = 0; j <= W; j++)
    dp[0][j] = 0;

for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= W; j++) {

        if (w[i] <= j) {
            dp[i][j] = max(
                v[i] + dp[i - 1][j - w[i]],
                dp[i - 1][j]
            );
        }
        else {

            dp[i][j] = dp[i - 1][j];
        }
    }
}

// — Print DP Table —
void printTable() {
    cout << "\nDP Table:" << endl;

    cout << "    ";
```



```
for (int j = 0; j <= W; j++)
    cout << j << " ";
cout << endl;
for (int i = 0; i <= n; i++) {
    cout << "i=" << i << " ";
    for (int j = 0; j <= W; j++)
        cout << dp[i][j] << " ";
    cout << endl;
}
}

// — Find Selected Items —
void findItems() {
    cout << "\nSelected Items:" << endl;
    int j = W;
    for (int i = n; i >= 1; i--) {
        if (dp[i][j] != dp[i - 1][j]) {
            cout << "Item " << i
                << " (w=" << w[i]
                << ", v=" << v[i]
                << ") TAKEN" << endl;
            j = j - w[i];
        }
        else {
            cout << "Item " << i
                << " (w=" << w[i]
                << ", v=" << v[i]
```



```
        << ") NOT taken" << endl;
    }
}
}
void showResult() {
    cout << "\nMaximum Value = "
        << dp[n][W] << endl;
}
};
int main() {
    Knapsack k;
    k.getInput();
    k.solve();
    k.printTable();
    k.showResult();
    k.findItems();
    return 0;
}
```



Output:

```
Enter number of items : 5
Enter knapsack capacity : 10
Item 1 weight and value : 2 3
Item 2 weight and value : 1 5
Item 3 weight and value : 3 6
Item 4 weight and value : 4 7
Item 5 weight and value : 5 6
```

DP Table:

```
    0 1 2 3 4 5 6 7 8 9 10
i=0  0 0 0 0 0 0 0 0 0 0 0
i=1  0 0 3 3 3 3 3 3 3 3 3
i=2  0 5 5 8 8 8 8 8 8 8 8
i=3  0 5 5 8 11 11 14 14 14 14 14
i=4  0 5 5 8 11 12 14 15 18 18 21
i=5  0 5 5 8 11 12 14 15 18 18 21
```

Maximum Value = 21

Selected Items:

```
Item 5 (w=5, v=6) NOT taken
Item 4 (w=4, v=7) TAKEN
Item 3 (w=3, v=6) TAKEN
Item 2 (w=1, v=5) TAKEN
Item 1 (w=2, v=3) TAKEN
```

=== Code Execution Successful ===



Time Complexity:

Handwritten code for 0/1 Knapsack problem time complexity analysis:

```
Time Complexity:-
for (int i=1; i<=n; i++) {
    for (int j=1; j<=w; j++) {
        if (w[i]>j) {
            dp[i][j] = max(
                v[i] + dp[i-1][j-w[i]],
                dp[i][j] = max(
                    v[i] + dp[i-1][j-w[i]],
                    dp[i][j] = dp[i-1][j];
        }
    }
}
```

$T(n, w) = n \times w \times O(1)$
 $T(n, w) = O(nw)$

Time Complexity is same for all 3 cases.

Conclusion: The 0/1 Knapsack problem was successfully implemented using Dynamic Programming. The algorithm calculates the maximum possible value within the given capacity and identifies the items that should be selected.